MedAssist-AI

Model Training, Evaluation, and Pipeline Optimization

Sanvi Sawant

1. Project Description

This project implements an end-to-end machine learning pipeline designed to automatically predict and classify human diseases based on complex patient symptom profiles. The primary goal is to process massive, high-cardinality tabular medical data to assist in rapid diagnostic predictions.

The system utilizes advanced gradient boosting architectures, rigorously benchmarking an optimized XGBoost model against LightGBM and Scikit-Learn's HistGradientBoosting frameworks to handle extreme multi-class classification natively without relying on pre-trained weights.

2. Dataset Used

The project utilizes a massive tabular dataset of patient medical records and symptom arrays.

- Original Scope: The dataset contains nearly 190,000 individual patient records.
- Classes: The data maps to 773 distinct, specific disease classes.
- Data Imbalance & Scaling: The dataset features severe class imbalance, including rare diseases represented by only a single row of patient data. The pipeline was engineered specifically to protect and learn from these low-frequency classes rather than scaling them out.

3. Environment Setup

The project was developed and executed using Google Colab to leverage cloud-based hardware acceleration. The implementation was written in Python. Key libraries and configurations include:

- **XGBoost & LightGBM:** The core gradient boosting frameworks utilized to build, train, and evaluate the decision tree architectures.
- **Scikit-Learn:** Essential machine learning library used for label encoding, data splitting, and benchmarking the HistGradientBoostingClassifier.
- **Pandas and NumPy:** Utilized for efficient numerical operations, tabular data handling, and processing the 190,000-row feature matrix.
- **Hardware Configuration:** To process the dataset efficiently, model training was split between hardware environments. LightGBM and HistGradientBoosting utilized maximum parallel CPU processing (n_jobs=-1), while the XGBoost architecture leveraged the Colab NVIDIA T4 GPU (device='cuda') for exponential speedup.

4. Data Exploration & Baseline Evaluation

Before preprocessing, the raw tabular data was analyzed to map the complexity of the symptom features and targets:

- **Target Verification:** Verified the distribution of the 773 disease classes, identifying the extreme sparsity and long-tail distribution of rare diseases.
- **Sanity Checks:** Confirmed the presence of single-row disease classes, which dictated the need for specialized data splitting and algorithm parameter tuning.

5. Data Preprocessing & Architecture Upgrades

The raw symptom dataset required strict standardization to be ingested by the gradient boosting models.

- **5.1 Label Encoding:** Scikit-Learn's LabelEncoder was utilized to translate the string-based disease names into a standardized numerical array format required for multi-class targets.
- **5.2 Custom Dataset Splitting:** Standard randomized train/test splits would have mathematically erased the single-row diseases. A custom splitting strategy was engineered to actively isolate and protect these low-frequency classes, ensuring they were present in the training set.
- **5.3 Histogram Binning:** Continuous and complex features were discretized into integer buckets (maximum 255 bins). This dropped the split-finding time complexity from $O(n \log n)$ to $O(n)$ and prevented out-of-memory (OOM) crashes.

6. Proposed System

The raw symptom dataset required strict standardization to be ingested by the gradient boosting models.

- **5.1 Label Encoding:** Scikit-Learn's LabelEncoder was utilized to translate the string-based disease names into a standardized numerical array format required for multi-class targets.
- **5.2 Custom Dataset Splitting:** Standard randomized train/test splits would have mathematically erased the single-row diseases. A custom splitting strategy was engineered to actively isolate and protect these low-frequency classes, ensuring they were present in the training set.
- **5.3 Histogram Binning:** Continuous and complex features were discretized into integer buckets (maximum 255 bins). This dropped the split-finding time complexity from $O(n \log n)$ to $O(n)$ and prevented out-of-memory (OOM) crashes.

7. Training Pipeline & Custom Evaluation

The final training phase was executed using 100 decision trees (estimators).

- **Execution:** The XGBoost architecture, accelerated by the T4 GPU, completed its training successfully in exactly 9.17 minutes.
- **Benchmarking Constraints:** Alternative LightGBM, Scikit-Learn were tested but failed to process the rare diseases due to strict default minimum-leaf constraints (requiring 20+ samples) and split-gain thresholds, causing them to abort tree building.

Results & Visualization:

```
================================================================================
                        FINAL MODEL COMPARISON (100 TREES)
================================================================================
            Model  Accuracy (%)  Precision (%)  Recall (%)  F1-Score (%)  Time (Mins)
          XGBoost         78.08          78.29       78.08         78.05         9.17
         LightGBM          0.57           0.01        0.57          0.02        21.52
Gradient Boosting          0.64           0.00        0.64          0.01        21.11
         LightGBM          0.50           1.72        0.50          0.12        20.86
```

Figure 1: Model Comparison Table

Final Evaluation Metrics (XGBoost):

- Overall Accuracy: 78.08%
- Overall Precision: 78.29%
- Overall Recall: 78.08%
- Overall F1-Score: 78.05%

8. Conclusion & Future Scope

This project successfully demonstrates a highly functional, end-to-end machine learning pipeline for complex tabular medical data. By engineering a custom preprocessing phase to protect rare diseases and leveraging GPU-accelerated histogram binning, the XGBoost architecture successfully mapped highly sparse symptom matrices. The system achieved over 78.08% accuracy across 773 distinct disease classes, proving the viability of the algorithm for large-scale, high-cardinality medical diagnostics.

Future Scope:

Hyperparameter Optimization: Implementing automated tuning frameworks (such as Optuna or GridSearchCV) to systematically adjust learning rates, maximum tree depth, and split-gain parameters to maximize predictive accuracy and prevent algorithmic collapse in CPU-bound frameworks like LightGBM.

Dataset Division & Profiling: Restructuring the monolithic 190,000-row dataset by clustering diseases based on their recurrence. Dividing the data into smaller, targeted sub-datasets (e.g., high-frequency vs. rare diseases) will reduce extreme class imbalance and computational overhead.